Nama: Hans Malvin Djojo

https://github.com/hansmalvin/LastMile

Step 1: Review Existing Schema
Subtask 1.1 Analyze Current Schema
Description:
Review the given existing schema (example: Order schema storing user details directly inside each order).
Example (Old Schema Concept):
Order contains name, email, address fields directly
No relationship between User and Order collection

Answer: In the original schema, user details like name and contact info were embedded directly into each order document. While this allowed for faster data retrieval, it lacked a formal link between the User and Order collections, leading to excessive data redundancy as the same information was saved across multiple records.

Subtask 1.2 Identify Design Issues
Description:
List issues in the old schema such as:
Data redundancy
Poor normalization
Difficult updates
Lack of validation
Validation Criteria:
At least 2–3 design problems identified
Clear reasoning provided

Answer:
1. Data Redundancy: Storing user details like names and emails within every order document leads to unnecessary storage consumption and bloated collections.
2. Update Anomalies: Modifying user information (e.g., a change of address) requires a massive "scatter-update" across all historical orders to maintain accuracy.
3. Poor Data Normalization: The lack of separation between User and Order entities violates standard database design principles, making the architecture less scalable.
4. Data Integrity Risks: Without a centralized user reference, there is no "source of truth," leading to potential inconsistencies and a lack of strict data validation.

Step 2: Plan New Schema Design
Subtask 2.1 Decide Relationship Strategy
Description:
Decide whether to use:
Embedded documents
O

Referenced documents (ObjectId with ref)

Answer: The new schema updates to a Referenced Relationship model using ObjectId. This approach was selected to:

Eliminate Data Redundancy: By referencing a single user document, we remove the need for duplicate data entries.

Simplify Maintenance: Updates to user profiles are now centralized, reflecting across all orders instantaneously.

Enhance Scalability: The decoupled structure allows the database to grow more efficiently.

Optimize 1: N Mapping: This design perfectly aligns with a one-to-many relationship, where a single user owns multiple order records.

Subtask 2.2 Design Improved Schema Structure
Description:
Redesign collections (example)
Create separate User schema
Order schema references User using ObjectId

Answer: User Collection
-name
-email

-address

2. Order Collection

-product

-quantity

-user (User menggunakan ObjectId)

Step 3: Implement New Schema
Subtask 3.1 Create Updated Schemas
Description:
Define new Mongoose schemas with improved structure.
Example improvements:
Use required validation
Use proper data types
Add timestamps

Answer:
const mongoose = require('mongoose');

```
const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: true
  },
  email: {
    type: String,
    required: true,
    unique: true
  },
  address: {
    type: String,
    required: true
  }
}, { timestamps: true });

// Order Schema
const orderSchema = new mongoose.Schema({
  product: {
    type: String,
    required: true
  },
  quantity: {
    type: Number,
    required: true
  },
  user: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  }
}, { timestamps: true });

const User = mongoose.model('User', userSchema);
const Order = mongoose.model('Order', orderSchema);

module.exports = { User, Order };
```

Subtask 3.2 Create Models
Description:
Create models for redesigned schemas.
Validation Criteria:
Models created successfully
No runtime errors

Answer:

```
const User = mongoose.model('User', userSchema);
const Order = mongoose.model('Order', orderSchema);
module.exports = { User, Order };
```

Step 4: Data Migration Simulation
Subtask 4.1 Insert Sample Data
Dscription:
Insert sample User and Order data using new structure.

Answer:
```
const user = await User.create({
name: "budi",
email: "budi@email.com",
address: "Bali"
});
const order = await Order.create({
product: "PC",
quantity: 10,
user: user._id
});
```

Subtask 4.2 Populate Referenced Data
Description:
Use .populate() to fetch related user data inside order query.

Answer:
```
const orders = await Order.find().populate('user');
console.log(orders);
```

Step 5: Test and Validate
Subtask 5.1 Test CRUD Operations
Description:
Perform basic create and read operations using new schema.

Answer:
1.Create: User and order data successfully added to the database
2.Read: Order data can display user information using function .populate()
3.No data duplication in the database

Subtask 5.2 Compare Old vs New Design
Description:
Compare the old schema and redesigned schema.
Focus on
Performance
Maintainability
Data consistency

Answer:

Organizational Structure

The Old Schema lacked clear boundaries, with user and order data stored together in a single, unseparated format. In contrast, the New Schema introduces a clean separation between the User and Order entities, allowing for more modular data management.

Redundancy and Storage

A major flaw of the Old Schema was high data redundancy, as user information was repeated across every order. The New Schema significantly lowers this redundancy by storing user details once and simply referencing them, leading to a much smaller storage footprint.

Ease of Data Updates

Updating information in the Old Schema was difficult and error-prone because a single change (like an address update) had to be manually applied to every related order. The New Schema makes updates easy; changing data in the User collection automatically reflects across all linked orders.

Consistency and Integrity

Data consistency was historically low in the Old Schema because there was no guarantee that the same user's data would match across different documents. The New Schema provides high consistency by establishing a single "source of truth" for every user.

Maintainability and Scalability

The Old Schema suffered from poor maintainability as the database grew, making it hard to manage and evolve. The New Schema offers a much better, future-proof structure that is easier to debug, audit, and scale.

Database Relationships

While the Old Schema had no formal relationships between data points, the New Schema utilizes ObjectId references. This creates a logical link between collections, enabling the database to handle complex queries and one-to-many relationships effectively.